



INTERNSHIP PROJECT ASSIGNMENT

ENTERPRISE LOGISTICS & WAREHOUSE MANAGEMENT PLATFORM (LOGIFLOW)

Full Stack Enterprise Project · 60-Day Advanced Program

Advanced Level · Industry Simulation · Logistics & Supply Chain

CANDIDATE NAME

Aryan Ajmera

REFERENCE NUMBER

NFGT/HR/INT/2026/160

INTERNSHIP MODE

Remote

INDUSTRY GUIDE

Aryan Sharma

DATE ISSUED

22 July 2026

EDUCATION

B.Sc

COLLEGE

University of Edinburgh, Noida (UP)

DURATION

22/07/2026 to 19/09/2026

REPORTING MANAGER

Sonia Sharma

PROJECT SEQUENCE

Second Project Allocation

NextGen Forge Technologies

ABIL Imperial Commercial Spaces, Baner, Pune, Maharashtra 411069

www.nextgenforgetechnologies.com | hr@nextgenforgetechnologies.com

[Ref: NFGT/HR/INT/2026/160](#)

01

INTERNSHIP ROLE OVERVIEW

This Internship Project Assignment has been officially issued by **NextGen Forge Technologies** to **Aryan Ajmera** as part of the Advanced Full Stack Software Development Internship Program. This is the **Second Project Allocation** within the internship engagement. The project is an advanced, enterprise-grade client simulation designed to provide real-world software engineering exposure across backend API development, React frontend integration, and cloud deployment.

The candidate will build the complete **LogiFlow Enterprise Logistics Operations & Warehouse Management Platform** — a full-featured, production-ready MERN stack application supporting five distinct user roles and fifteen operational modules. Every task is calibrated at **3–5 hours per day, Monday to Friday**, across 60 days using only free-tier services.

CANDIDATE NAME:

Aryan Ajmera

REFERENCE NUMBER:

NFGT/HR/INT/2026/160

DOMAIN:

Full Stack Software Development

MODE:

Remote

DURATION:

60 Days (8 Weeks)

PROJECT SEQUENCE:

Second Project Allocation

INDUSTRY GUIDE:

Aryan Sharma

REPORTING MANAGER:

Sonia Sharma

Internship Focus Areas:

- **Enterprise RBAC Architecture:** Implement five user roles (Administrator, Warehouse Manager, Warehouse Executive, Logistics Coordinator, Customer Support Executive) with module-level access control using JWT and middleware.
- **Full Logistics Domain Coverage:** Build 15 end-to-end modules including Inventory, Shipment Lifecycle, Dispatch Planning, Vehicle & Driver Management, and Support Tickets.
- **Advanced MongoDB Design:** Design relational-style document schemas with references, aggregation pipelines, and indexed queries for analytics.
- **Production Cloud Deployment:** Deploy the frontend on Vercel, backend on Render, and database on MongoDB Atlas — all using free tiers.
- **Professional Engineering Practices:** Structured API documentation, weekly GitHub submissions, and bi-weekly mentor reviews.

02

PROJECT DESCRIPTION

PROJECT: LogiFlow - Enterprise Logistics Operations & Warehouse Management Platform

Advanced Level

LogiFlow is an enterprise-grade web application designed to digitize and streamline logistics and warehouse operations. The platform provides a centralized system to manage warehouses, inventory, customer orders, shipment processing, dispatch planning, and delivery tracking through a single integrated application.

The application is intended for internal business operations and simulates the type of software used by logistics companies, third-party logistics (3PL) providers, courier services, and warehouse management organizations. The system follows a modular architecture where each module communicates through secure REST APIs while maintaining a centralized MongoDB database.

Platform Capabilities:

- Five-role RBAC (Administrator, Warehouse Manager, Warehouse Executive, Logistics Coordinator, Customer Support Executive).
- Complete shipment lifecycle management from order creation to final delivery confirmation.
- Real-time dispatch planning with vehicle and driver assignment engine.
- Centralized analytics dashboard with MongoDB aggregation-powered KPIs and Chart.js visuals.
- Activity logs, in-app notification system, and support ticket management.
- Exportable operational reports in PDF and CSV format for management and auditing.

03

PROBLEM STATEMENT

Modern logistics organizations process thousands of shipments daily across multiple warehouses. As businesses expand, fragmented tools — spreadsheets, paper-based records, and disconnected software — create dangerous operational bottlenecks that reduce fulfillment speed, increase errors, and limit management visibility.

Absence of Role-Based Operational Control

Without a centralized RBAC system, multiple staff categories operate on shared data without access boundaries, causing integrity failures and unauthorized modifications.

Fragmented Inventory & Warehouse Visibility

Warehouse executives maintain inventory on disconnected spreadsheets. Real-time stock levels are unavailable to logistics coordinators, causing order fulfillment failures.

Manual Dispatch Operations

Dispatch managers manually match shipments to vehicles without real-time fleet availability data, creating costly delays and driver idle time.

No Unified Customer & Support Layer

Customer support executives have no centralized system to view order status, raise tickets, or communicate resolutions — resulting in poor client satisfaction metrics.

The **LogiFlow Platform** addresses all of these challenges through a modular, role-protected, API-driven enterprise application with real-time dashboards, an integrated ticketing system, and a full reporting engine.

04

PROJECT FEASIBILITY

This project is specifically designed for final-year B.Tech and MCA students with intermediate web development experience. At **3–5 hours per day, Monday to Friday (~15–25 hours/week)**, each weekly task block is completable within its allocated timeframe across 8 weeks.

TARGET CANDIDATES:

B.Tech Final Year, MCA, and other advanced IT Candidates

DIFFICULTY LEVEL:

Advanced — Enterprise Client Simulation

WORKING COMMITMENT:

3–5 Hours/Day · Mon–Fri · ~15–25 Hours/Week

WEEKLY STRUCTURE:

2 Days (Task 1) + 2 Days (Task 2) + 1 Day (Integration/Review)

TIMELINE:

8 Weeks · 24 Task Blocks · 60 Days

FREE TIER STACK:

MongoDB Atlas · Render · Vercel — Zero cost for development

Why this project is feasible at 3–5hr/day:

- **MERN Stack Consistency:** JavaScript across the entire stack eliminates language-switching overhead, allowing the candidate to build momentum from Week 1.
- **Free Infrastructure:** MongoDB Atlas (512MB free), Render (free web service tier), and Vercel (free Hobby tier) require zero financial investment.
- **Modular 2+2+1 Structure:** Each week is divided into two 2-day focused tasks followed by a 1-day integration and review session — preventing context overload.
- **Progressive Complexity:** Month 1 focuses entirely on backend APIs. Month 2 shifts to frontend UI and deployment. Each phase builds directly on the previous.
- **No Research-Level Tasks:** Every task implements practical, well-documented patterns from the official Express, React, and MongoDB documentation.

05

TECHNOLOGIES & TOOLS REQUIRED

The following technology stack aligns with industry-standard enterprise MERN development. All tools offer free tiers suitable for development, testing, and deployment.

Technology	Category	Purpose
React.js	Frontend Framework	Component-based SPA for all role-specific dashboards and module UIs.
Tailwind CSS	UI Styling	Utility-first CSS enabling rapid, responsive, professional UI development.
React Router v6	Client Routing	Declarative routing for protected routes and role-based navigation.
Axios	HTTP Client	Promise-based client for all API calls between frontend and backend.
React Hook Form	Form Management	Performant form state management with built-in validation.

Technology	Category	Purpose
Node.js	Runtime Environment	JavaScript runtime for the Express backend and business logic layer.
Express.js	Backend Framework	Lightweight framework for building all REST APIs with middleware.
JWT	Authentication	JSON Web Token standard for stateless, role-encoded authentication.
bcrypt	Security	Cryptographic hashing for secure password storage and verification.
Express Validator	Input Validation	Server-side validation middleware for all API request payloads.
Multer	File Uploads	Multipart form data middleware for document and image upload support.
MongoDB Atlas	Database (Cloud)	Serverless MongoDB cluster — 512MB free tier for production use.
Mongoose	ODM	Schema-based data modelling with validation and relationship support.
Vercel	Frontend Hosting	Free-tier hosting for the React SPA with automatic CI/CD from GitHub.
Render	Backend Hosting	Free-tier hosting for the Express.js backend web service.
Postman	API Testing	API client for testing and documenting all REST endpoints.

06

KEY FEATURES TO IMPLEMENT

The following 15 feature modules constitute the complete functional scope of the LogiFlow Platform. All modules must be implemented, API-tested, and UI-integrated before final submission.

#	Module	Description
01	User Authentication & Authorization	Secure register/login using JWT and bcrypt. Token expiry, refresh logic, and session management.
02	Role-Based Access Control	Module-level access restriction for 5 roles via Express middleware and React frontend route guards.
03	Warehouse Management	CRUD for warehouse records including location, capacity, operational status, and assigned manager.
04	Product Management	Product catalogue management with categories, SKUs, unit-of-measure, and product status.
05	Inventory Management	Real-time stock tracking per warehouse with restock triggers and low-stock threshold alerts.
06	Customer Management	CRM module for client onboarding, contact details, credit terms, and account status management.
07	Order Management	Order lifecycle from Draft through Confirmed, Processing, Dispatched, Delivered, and Cancelled.

#	Module	Description
08	Shipment Lifecycle	End-to-end shipment tracking with appended tracking events from creation to delivery confirmation.
09	Vehicle & Driver Management	Fleet registry with vehicle availability, maintenance status, capacity, and driver licence records.
10	Dispatch Planning	Assignment engine linking shipments to vehicles and drivers with scheduling and routing notes.
11	Dashboard & Analytics	Role-specific KPI dashboards powered by MongoDB aggregation pipelines with Chart.js visuals.
12	Activity Logs	Immutable audit trail capturing all create, update, and delete operations with user and timestamp.
13	Notifications	In-app notification system for order status changes, low-stock alerts, and dispatch confirmations.
14	Support Ticket Management	Internal ticketing module for customer issue logging, assignment, escalation, and resolution.
15	Reports & Data Export	Filterable operational reports exportable as PDF and CSV for management and auditing.

07

60-DAY WEEKLY TASK BREAKDOWN — 8-WEEK SCHEDULE

Structured across 2 monthly phases with 4 weekly task blocks each. Each week follows a **2 Days + 2 Days + 1 Day** pattern: two focused 2-day development tasks followed by a dedicated integration and review session (1 day). At 3–5 hours/day, Mon–Fri, every task is achievable within its allocated week. Complexity increases progressively from foundational backend setup to advanced frontend integration and production deployment.

MONTH 1 — BACKEND ENGINEERING & REST APIs (Weeks 1–4)

WEEK 1

~ 3 tasks · 2 days, 2 days, 1 day

Task W1.1: Project Setup, Architecture & MongoDB Schema Design (Days 1–2)

Description:	Initialize the Node.js/Express server with full MVC folder structure: routes/, controllers/, models/, middleware/, config/, utils/. Connect to MongoDB Atlas using Mongoose. Design and implement all 12 Mongoose schemas: User, Role, Warehouse, Product, Inventory, Customer, Order, OrderItem, Shipment, ShipmentEvent, Vehicle, Driver, Dispatch, Ticket, Notification, ActivityLog. Define all fields, data types, validators, enum values, and inter-document ObjectId references. Configure dotenv for environment variables. Initialize a Git repository with main and develop branches. Add a .gitignore excluding node_modules and .env. Document the complete schema design in README.md with a table listing all models and their key fields.
Deliverables:	GitHub repo (main + develop branches), 12+ Mongoose models, MongoDB Atlas connected, .env configured, schema README

Task W1.2: JWT Authentication System & RBAC Middleware (Days 3–4)

Description:	Implement the complete authentication layer. Create POST /api/auth/register (accept name, email, password, role; hash password with bcrypt saltRounds=12; save User document). Create POST /api/auth/login (verify credentials; sign JWT with payload: userId, email, role; set token expiry to 8h). Create POST /api/auth/logout (optional: server-side token blacklist using an in-memory Set or a BlacklistedToken model). Write verifyToken middleware: extract Bearer token from Authorization header, verify signature, attach decoded payload to req.user. Write authorizeRoles(...allowedRoles) middleware: check req.user.role against allowedRoles array; return 403 if not permitted. Write a rolePermissions.js config mapping each of the 5 roles to their permitted module-level operations. Test all auth endpoints in Postman. Verify JWT decoding and role rejection with tokens of different roles.
Deliverables:	POST /api/auth/(register, login, logout), verifyToken middleware, authorizeRoles middleware, rolePermissions config, Postman tests

Task W1.3: Week 1 Integration & Review (Day 5)

Description:	Apply verifyToken and authorizeRoles middleware to 2 sample protected routes covering 2 different roles and verify correct accept/reject behavior. Insert 5 test users covering all 5 roles into the database via Postman. Verify Mongoose schema validators reject invalid documents. Commit all work to the develop branch with a clear commit message. Open a GitHub PR from develop to main for Week 1 work. Write and submit the Week 1 progress report to hr@nextgenforgetechnologies.com.
Deliverables:	Middleware tested on protected routes, 5 test users inserted, schema validators verified, GitHub PR, Week 1 report

WEEK 2

~ 3 tasks · 2 days, 2 days, 1 day

Task W2.1: Warehouse & Product Management APIs (Days 1–2)

Description:	Implement Warehouse Management APIs: POST /api/warehouses (create warehouse with name, location, city, totalCapacity, managerId, status). GET /api/warehouses (paginated list; query params: city, status, page, limit). GET /api/warehouses/:id (full detail). PUT /api/warehouses/:id (update fields). DELETE /api/warehouses/:id (soft delete: set status to 'Inactive'). Restrict create/update/delete to Administrator and Warehouse Manager roles; all roles can GET. Add Express Validator schema validation for all create/update endpoints. Next, implement Product Management APIs: POST /api/products (name, sku, category, unit, description, status). GET /api/products (filter by category, search by name/sku, pagination). GET /api/products/:id. PUT /api/products/:id. DELETE /api/products/:id (soft delete). Restrict create/update/delete to Administrator and Warehouse Manager. Test all 10 endpoints in Postman with valid, invalid, and unauthorized request scenarios.
Deliverables:	Warehouse APIs (5 endpoints + role restrictions), Product APIs (5 endpoints + role restrictions), Express Validator schemas, Postman collection updated

Task W2.2: Inventory Management APIs (Days 3–4)

Description:	Implement Inventory Management APIs. POST /api/inventory (create inventory record linking warehouseId + productId + quantity + reorderThreshold + unit). GET /api/inventory (filter by warehouseId, productId, low-stock flag; pagination). GET /api/inventory/:id (detail). PUT /api/inventory/:id/restock (increment quantity by restockAmount; log activity). PUT /api/inventory/:id/deduct (decrement quantity by deductionAmount; validate quantity cannot go negative; if quantity falls below reorderThreshold, set lowStockAlert: true; log activity). GET /api/inventory/low-stock (list all inventory records where lowStockAlert is true). Implement business logic: prevent stock deduction below zero; auto-set lowStockAlert flag on deduction. Create a utility function triggerLowStockNotification() that creates a Notification document when low-stock is flagged. Test all endpoints including edge cases (negative stock, zero stock, over-restock).
Deliverables:	Inventory APIs (6 endpoints), no-negative-stock logic, lowStockAlert auto-flag, Notification trigger, edge-case test results

Task W2.3: Week 2 Integration & Review (Day 5)

Description:	Test end-to-end flow: Create Warehouse -> Create Product -> Create Inventory record -> Deduct stock -> Verify lowStockAlert flag set -> Verify Notification document created. Ensure all role restrictions are correctly applied across Warehouse, Product, and Inventory endpoints. Fix any Mongoose reference validation errors. Commit all changes to develop branch. Write and submit Week 2 progress report.
Deliverables:	End-to-end inventory flow verified, role restrictions confirmed, low-stock notification tested, GitHub commit, Week 2 report

WEEK 3

~ 3 tasks · 2 days, 2 days, 1 day

Task W3.1: Customer & Order Management APIs (Days 1–2)

Description:	Implement Customer Management APIs: POST /api/customers (companyName, contactPersonName, email, phone, address, creditLimit, accountStatus). GET /api/customers (search by name/email, filter by status, pagination). GET /api/customers/:id (full profile with order count summary). PUT /api/customers/:id (update). DELETE /api/customers/:id (deactivate: set accountStatus to 'Inactive'). Restrict create/delete to Administrator and Logistics Coordinator. Implement Order Management APIs: POST /api/orders (create order: customerId, warehouseId, array of orderItems [{productId, quantity, unitPrice}]; validate stock availability from Inventory; calculate totalAmount; set status to 'Draft'; reserve stock using inventory deduct). GET /api/orders (filter by status, customerId, date range, pagination). GET /api/orders/:id (full detail with customer info and order items). PUT /api/orders/:id/status (status machine: Draft -> Confirmed -> Processing -> Dispatched -> Delivered -> Cancelled; implement forward-only validation; on Cancellation, return stock via inventory restock). Create ActivityLog document on every order status change.
Deliverables:	Customer APIs (5 endpoints), Order APIs (5 endpoints + status machine), stock reservation on confirm, stock return on cancel, ActivityLog

Task W3.2: Shipment Lifecycle Management APIs (Days 3–4)

Description:	Implement Shipment APIs. POST /api/shipments (auto-created from a Confirmed order: orderId, originWarehouseId, destinationAddress; auto-generate trackingId using uuid/nanoid; set status to 'Created'; link shipment back to order). GET /api/shipments (filter by status, warehouseId, date range, pagination). GET /api/shipments/:id (full detail including full tracking event history). PUT /api/shipments/:id/status (status machine: Created -> Assigned -> In-Transit -> Out-for-Delivery -> Delivered -> Failed; on Delivered: auto-update linked Order status to Delivered; on Failed: trigger Notification to Logistics Coordinator). POST /api/shipments/:id/tracking (append a ShipmentEvent: location, description, timestamp; push to shipment trackingHistory array). Implement the business logic cross-linking: updating Shipment to Delivered auto-updates linked Order. Test the full shipment lifecycle in Postman from Created to Delivered including the Order auto-update.
Deliverables:	Shipment APIs (5 endpoints + status machine), tracking event append, order-status auto-sync, Delivered/Failed notifications, Postman lifecycle test

Task W3.3: Week 3 Integration & Review (Day 5)

Description:	Run end-to-end test: Create Customer -> Create Order -> Confirm Order (stock deducted) -> Create Shipment -> Progress to Delivered -> Verify Order auto-updates to Delivered. Test Order Cancellation (stock returned). Verify ActivityLog entries for all status changes. Fix any cross-module reference issues. Commit to develop branch. Write and submit Week 3 progress report.
Deliverables:	Complete Order-to-Delivery flow verified, ActivityLog entries confirmed, cancellation stock-return tested, GitHub commit, Week 3 report

WEEK 4

~ 3 tasks · 2 days, 2 days, 1 day

Task W4.1: Vehicle, Driver & Dispatch Planning APIs (Days 1–2)

Description:	Implement Vehicle Management APIs: POST /api/vehicles (licensePlate, type, capacityKg, status: Available/In-Use/Maintenance). GET /api/vehicles (filter by status, type; pagination). GET /api/vehicles/available (return only Available vehicles in real-time). PUT /api/vehicles/:id/status (update status). DELETE /api/vehicles/:id. Implement Driver Management APIs: POST /api/drivers (name, licenceNumber, phone, assignedVehicleId, available: true). GET /api/drivers (filter by availability). PUT /api/drivers/:id/availability. DELETE /api/drivers/:id. Implement Dispatch Planning APIs: POST /api/dispatches (shipmentId, vehicleId, driverId, scheduledDate, routeNotes; validate: vehicle must be Available, driver must be available; on creation set vehicle status to In-Use, driver available to false; set shipment status to Assigned). GET /api/dispatches (filter by date, status; pagination). GET /api/dispatches/:id (full detail). PUT /api/dispatches/:id/complete (set vehicle back to Available, driver available to true; set shipment status to Delivered). Add availability validation that rejects In-Use vehicles and unavailable drivers with descriptive error messages.
Deliverables:	Vehicle APIs (5), Driver APIs (4), Dispatch APIs (4 + availability validation), availability state changes on create/complete, Postman tests

Task W4.2: Analytics, Activity Logs, Notifications & Support Ticket APIs (Days 3–4)

Description:	Implement Dashboard Analytics: GET /api/analytics/summary using MongoDB aggregation pipelines computing: totalActiveOrders, shipmentsByStatus (array), warehouseUtilisation (array: warehouseId, usedCapacityPercent), topProductsByOrderVolume (top 5), dispatchesCompletedThisWeek. GET /api/analytics/orders-by-day (last 30 days order count per day for line chart). Implement Activity Logs: ActivityLog schema already created; expose GET /api/logs (admin-only; filter by entity, userId, date range; pagination). Implement Notifications: GET /api/notifications/me (return current user's unread notifications, newest first). PATCH /api/notifications/:id/read (mark single notification as read). PATCH /api/notifications/read-all (mark all user notifications as read). Verify notification auto-creation from Week 2 (low-stock) and Week 3 (shipment failed) triggers still works. Implement Support Ticket APIs: POST /api/tickets (title, description, priority: Low/Medium/High/Critical, linkedOrderId optional). GET /api/tickets (filter by status, priority; pagination; admin sees all, support exec sees assigned). PUT /api/tickets/:id/assign (assigneeId; only Admin). PUT /api/tickets/:id/status (Open -> In-Progress -> Resolved -> Closed). POST /api/tickets/:id/comment (append comment to ticket timeline array). Test all endpoints. Export full Postman collection as JSON.
Deliverables:	Analytics aggregation APIs (2 endpoints), Activity Log GET, Notification APIs (3 endpoints), Ticket APIs (5 endpoints + comment), Postman collection exported

Task W4.3: Week 4 Backend Integration Review (Day 5)

Description:	Simulate all 5 user roles: register and login each role, obtain tokens, and run the primary business flow from each role's perspective. Verify every role's access permissions work correctly across all 15 module APIs. Verify all ActivityLog, Notification, and analytics data is accurate after the flows. Write comprehensive API reference documentation in README.md covering all endpoints, request/response schemas, and role restrictions. Commit final backend code to main branch via PR. Write and submit Week 4 progress report.
Deliverables:	All 5-role permission simulation complete, analytics data verified, comprehensive API documentation written, GitHub PR to main, Week 4 report

MONTH 2 — FRONTEND DEVELOPMENT & CLOUD DEPLOYMENT (Weeks 5–8)

WEEK 5

~ 3 tasks · 2 days, 2 days, 1 day

Task W5.1: React App Setup, Authentication UI & Dashboard Shell (Days 1–2)

Description:	Initialize the React application using Vite (npm create vite@latest). Install and configure Tailwind CSS v3. Install dependencies: react-router-dom v6, axios, react-hook-form, recharts. Create a configured Axios instance (src/api/axios.js) with base URL from VITE_API_BASE_URL env variable and a request interceptor that automatically attaches the JWT Authorization header from localStorage. Build Authentication screens: Login page (email/password form using React Hook Form with client-side validation, error display, loading state, role-aware redirect on success) and a 404 Not Found page. Implement AuthContext using React createContext and useReducer: store user object, token, and role; expose login() and logout() actions; persist token to localStorage; provide the context to the entire application. Build ProtectedRoute component (wrapper around React Router Outlet) that redirects unauthenticated users to /login. Build the Application Shell: persistent left sidebar with branding logo; role-conditional menu items (each menu item checks the user's role before rendering); top navigation bar with user avatar, role badge, and notification bell icon with unread count badge; main content area with page title and breadcrumb.
Deliverables:	React/Vite app, Tailwind configured, Axios instance with JWT interceptor, Login screen, AuthContext, ProtectedRoute, full Dashboard shell with role-conditional sidebar

Task W5.2: KPI Dashboard, Analytics Charts & Notification Panel (Days 3–4)

Description:	Build the main Dashboard page by fetching data from GET /api/analytics/summary and GET /api/analytics/orders-by-day. Build 6 KPI Metric Cards component: Total Orders (this month), Active Shipments, Available Vehicles, Low-Stock Items, Open Tickets, Completed Dispatches — each card displays the metric value, label, a color-coded status indicator, and a subtle trend arrow. Install recharts. Implement a Doughnut chart for Shipments by Status breakdown. Implement a Bar chart for Top 5 Products by Order Volume. Implement a Line chart for Orders Created per Day (last 30 days). Build a Recent Activity feed component fetching the 5 most recent ActivityLog entries and displaying them as a timeline. Implement role-specific dashboard: Warehouse Manager sees Inventory/Warehouse KPIs; Logistics Coordinator sees Order/Dispatch KPIs; Customer Support sees Ticket KPIs — achieved by conditionally rendering KPI cards based on AuthContext role. Build the Notification dropdown panel: clicking the bell icon in the top nav opens a dropdown listing the 10 most recent unread notifications; each notification has a mark-as-read button; a 'Mark All Read' button at the top. Fetch from GET /api/notifications/me and PATCH /api/notifications/read-all.
Deliverables:	6 KPI cards, Doughnut/Bar/Line recharts, Recent Activity feed, role-specific dashboard views, Notification dropdown panel with mark-as-read

Task W5.3: Week 5 Integration & Review (Day 5)

Description:	Test role-specific navigation: log in with each of the 5 role tokens, verify the correct sidebar menu items appear, correct KPI cards render, and incorrect routes are blocked by ProtectedRoute. Test Notification dropdown: trigger a low-stock notification from Postman and verify it appears in the panel. Fix any Axios interceptor issues or auth state persistence bugs. Commit React app to GitHub (frontend/ directory). Write and submit Week 5 progress report.
Deliverables:	All 5 roles tested for correct navigation and KPI rendering, route protection verified, notification panel tested, GitHub commit, Week 5 report

WEEK 6

~ 3 tasks · 2 days, 2 days, 1 day

Task W6.1: Warehouse, Product, Inventory & Customer Management UI (Days 1–2)

Description:	Build Warehouse Management screens: a data table page with columns (Name, City, Capacity, Status, Manager) with search box and status filter dropdown, pagination controls, and an Edit action button per row. Add Warehouse modal with a React Hook Form form (name, location, city, totalCapacity, status) with client-side validation and Axios POST request. Edit Warehouse modal pre-populated with existing data using Axios PUT. A Warehouse Detail page listing linked inventory items. Build Product Management screens: product catalogue table with category filter, search by name/SKU, Add Product and Edit Product modals. Build Inventory Management screens: inventory stock table with warehouse and product name columns, a quantity badge component (green $\geq$ threshold, orange at threshold, red below threshold), Restock Action button (opens modal with quantity input, submits to PUT <code>/api/inventory/:id/restock</code>), a Low-Stock filter toggle to show only flagged items. Build Customer Management screens: customer directory table with search by name/email/phone and status filter, a Customer Detail page (full profile, contact info, account status, linked order count), Add Customer and Edit Customer modals using React Hook Form. Connect all screens to their respective backend APIs with Axios, implement loading spinners, and display API error messages using a toast notification component.
Deliverables:	Warehouse UI (table, add/edit modals, detail page), Product UI, Inventory UI (color badges, restock modal, low-stock filter), Customer UI (directory, detail, add/edit)

Task W6.2: Order Management UI (Days 3–4)

Description:	Build Order Management screens. Order List page: table with columns (Order ID, Customer, Total Amount, Status, Created Date) filterable by status dropdown and date range pickers; search by customer name; pagination. Order Detail page: displays customer info card, ordered items table (product name, quantity, unit price, line total), total amount summary, current status badge, and a vertical Order Status Timeline component visualizing the progression from Draft to Delivered with timestamps. Create Order Wizard: a multi-step form (Step 1: select customer from a searchable dropdown; Step 2: add order items by selecting product from dropdown and entering quantity — validate against current inventory stock; Step 3: review order summary with calculated totals and confirm). Order Action Buttons: Confirm button (only visible on Draft orders, only for Logistics Coordinator and Admin) with a confirmation dialog; Cancel button (with stock-return warning dialog). Status update calls PUT <code>/api/orders/:id/status</code> and refreshes the detail page. Implement optimistic UI updates on status change.
Deliverables:	Order list (filterable), Order detail with status timeline, Create Order wizard (3-step), Confirm/Cancel action buttons with dialogs, optimistic status updates

Task W6.3: Week 6 Integration & Review (Day 5)

Description:	Test the complete Customer-to-Order frontend flow: Onboard Customer -> Create Order (add 3 items) -> Verify inventory stock visually decreases on Inventory screen -> Confirm Order -> Verify status timeline updates. Test Order Cancellation and verify stock visually restores. Test form validation errors on all forms (empty required fields, invalid SKU, negative quantity). Commit to GitHub. Write and submit Week 6 progress report.
Deliverables:	Customer-to-Order UI flow tested end-to-end, inventory stock visual update verified, form validations confirmed, GitHub commit, Week 6 report

WEEK 7

~ 3 tasks · 2 days, 2 days, 1 day

Task W7.1: Shipment Tracking, Dispatch, Fleet Management & Ticket UI (Days 1–2)

Description:	Build Shipment Management screens: Shipment List table (Tracking ID, Order ref, Origin Warehouse, Status, Scheduled Date) with status filter and date range filter. Shipment Detail page with: order reference card, destination address, assigned vehicle/driver info panel, and a vertical Tracking History Timeline (each event shows location, description, and timestamp in chronological order). Status Action Buttons: Assign (opens Dispatch Assignment form), Mark In-Transit, Mark Out-for-Delivery, Mark Delivered, Mark Failed — each visible only to permitted roles and valid statuses. Build Vehicle Management screens: fleet table with license plate, type, capacity, and color-coded availability status badge (green: Available, orange: In-Use, grey: Maintenance). Add/Edit Vehicle modals. Build Driver Management screens: driver directory table with name, licence, phone, and availability toggle. Add/Edit Driver modals. Build Dispatch Planning screens: Dispatch Assignment modal (select pending Shipment -> select Available Vehicle from dropdown -> select Available Driver from dropdown -> set scheduled date -> add route notes -> confirm; disables vehicle/driver options that are already In-Use/unavailable). Dispatch List table with filters for date and status. Dispatch Detail page with a Complete Dispatch action button. Build Support Ticket screens: Ticket List (filter by status, priority, assigned to me toggle). Ticket Detail (full timeline of status changes and comments). Create Ticket modal. Assign and Resolve action buttons.
Deliverables:	Shipment UI (list + detail + tracking timeline + action buttons), Vehicle UI, Driver UI, Dispatch UI (assignment modal, list, detail + complete button), Ticket UI

Task W7.2: Reports UI, Activity Log Viewer & Dashboard Enhancements (Days 3–4)

Description:	Build the Reports & Export page: a report generation UI with a report type selector (Shipment Summary, Order History, Inventory Status Report, Dispatch Log), date range pickers (from/to), and a warehouse filter. A Generate Report button fetches data from the respective analytics or list endpoints with the given filters. Display the fetched data in an HTML table preview on the page. Implement PDF export using jsPDF + html2canvas: capture the table preview div and download as a formatted PDF. Implement CSV export using a custom utility function that converts the JSON array to a comma-separated string and triggers a browser download. Build the Activity Log viewer page (Admin only): paginated table showing all activity log entries (entity type, action, performed by, timestamp) with filters for entity type, user, and date range. Enhance the main Analytics Dashboard: add Warehouse Utilisation horizontal bar chart (fetched from analytics/summary.warehouseUtilisation). Add an Active vs Completed Dispatches count card. Ensure all new pages are added to the sidebar menu with appropriate role restrictions.
Deliverables:	Reports UI (type selector, date filter, table preview, PDF export, CSV export), Activity Log viewer (admin only), Warehouse Utilisation chart, updated sidebar

Task W7.3: Week 7 Integration & Review (Day 5)

Description:	Test the complete Dispatch flow from the React frontend: Confirm Order -> Create Shipment (via Postman) -> Assign Dispatch from frontend -> Verify vehicle shows as In-Use on fleet page -> Mark Dispatch Complete -> Verify vehicle returns to Available -> Verify Shipment and Order show as Delivered. Test PDF and CSV export with a date-filtered report. Test support ticket create, assign, and resolve flow. Commit to GitHub. Write and submit Week 7 progress report.
Deliverables:	Full Dispatch-to-Delivery UI flow tested, vehicle availability state change verified, PDF/CSV export tested, ticket lifecycle confirmed, GitHub commit, Week 7 report

WEEK 8

~ 3 tasks · 2 days, 2 days, 1 day

Task W8.1: End-to-End Integration Testing & Bug Resolution (Days 1–2)

Description:	Conduct comprehensive end-to-end testing covering all 15 modules and all 5 user roles from the React frontend. For each of the 5 roles, log in and execute the primary business workflows relevant to that role: Administrator (user management, all modules accessible, activity log review), Warehouse Manager (warehouse CRUD, inventory restock, low-stock review), Warehouse Executive (inventory deduction, shipment tracking), Logistics Coordinator (order creation, dispatch assignment, shipment management), Customer Support Executive (ticket creation, assignment, resolution). Test the full order-to-delivery lifecycle at least 3 times with different product combinations. Test boundary cases: zero-stock order creation attempt, duplicate SKU registration, expired JWT handling (should redirect to login), and unauthorized route access. Maintain a structured bug log documenting every issue found with: Module, Description, Steps to Reproduce, Severity (Critical / High / Medium / Low), and Resolution Status. Resolve all Critical and High severity bugs before proceeding. Perform basic cross-browser testing on Chrome, Firefox, and Microsoft Edge.
Deliverables:	Full 5-role business flow test results, structured bug log (all Critical + High resolved), 3 Order-to-Delivery lifecycle tests, cross-browser test results

Task W8.2: Cloud Deployment: Vercel + Render + MongoDB Atlas (Days 3–4)

Description:	Prepare and execute the complete production deployment. MongoDB Atlas: verify Atlas cluster is active on the free M0 tier; create a dedicated production database user with readWrite permission on the production database; whitelist 0.0.0.0/0 (allow all IPs) for Render backend access; copy the production MONGODB_URI connection string. Render Backend Deployment: connect the GitHub repository to a new Render Web Service; set the root directory to backend/; configure the build command as npm install and the start command as node index.js or node server.js; add all environment variables (MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN, NODE_ENV=production, CORS_ORIGIN pointing to Vercel URL); trigger a manual deploy; verify the live backend health endpoint. Vercel Frontend Deployment: connect the GitHub repository to a new Vercel project; set the root directory to frontend/; configure VITE_API_BASE_URL environment variable to the Render backend URL; trigger a deploy; verify the live frontend URL. Post-deployment validation: test all 5 login flows on the live app; run the Order-to-Delivery lifecycle on the live production environment; verify all charts load with real data.
Deliverables:	Live Render backend URL (tested), Live Vercel frontend URL (tested), all 5 roles verified on production, Order-to-Delivery lifecycle on live environment

Task W8.3: Final Documentation & Project Submission (Day 5)

Description:	Prepare the complete project submission package. Write a comprehensive README.md covering: project overview and objectives, system architecture description (frontend/backend/database separation), complete API endpoint reference table (method, path, description, required role for all endpoints), environment variable reference guide, local development setup instructions (step-by-step for both frontend and backend), and live deployment URLs with access credentials for testing. Clean the GitHub repository: ensure the main branch contains the final production code; remove all debug console.log and console.error statements; verify .gitignore is excluding .env files; write meaningful final commit messages. Write a project report document (2–3 pages) covering: architecture decisions made, challenges encountered and solutions applied, what was learned, and suggestions for future feature expansion. Submit the final package via email to hr@nextgenforgetechnologies.com including: GitHub repository link, live Vercel URL, live Render API URL, and the project report as a PDF attachment.
--------------	--



Deliverables:	Comprehensive README with API reference, clean GitHub repository (main branch), project report (2-3 pages), final submission email sent
---------------	---

08

CANDIDATE RESPONSIBILITIES

As a NextGen Forge Technologies intern, **Aryan Ajmera** is expected to maintain the professional standards outlined below throughout the 60-day engagement.

Optimised & Maintainable Code	Write clean, well-structured, documented JavaScript code following RESTful design standards. Prioritise readability, modularity, and maintainability across all Express and React modules. No hardcoded credentials or API keys in committed code.
Meeting Deadlines	Complete all weekly task blocks within the allocated 5-day timeframe. Communicate blockers to the Industry Guide at least 24 hours in advance with a written explanation and proposed resolution timeline.
Testing & Debugging	Independently test all API endpoints via Postman and all React UI flows before each weekly GitHub commit. Maintain a structured bug log documenting all issues, severity ratings, root causes, and resolution notes.
Documentation Standards	Maintain inline code comments and function-level documentation. Submit weekly progress reports every Friday by 6:00 PM IST to hr@nextgenforgetechnologies.com covering tasks completed, blockers faced, and plan for next week.
Progress Reviews	Participate actively in all bi-weekly mentor reviews. Come prepared with a live demo of completed features, a progress summary document, and a clear articulation of next steps and any open technical blockers.
Professional Communication	Communicate respectfully with the Industry Guide and Reporting Manager via official channels. Respond to all email and WhatsApp Business queries within 24 working hours.
Professional Conduct	Maintain strict confidentiality of all project materials including API keys, database credentials, business logic, and platform architecture. Represent NextGen Forge Technologies with professional integrity throughout the engagement.

09

MENTORSHIP & REPORTING STRUCTURE

NextGen Forge Technologies provides a structured mentorship framework to support the candidate with technical guidance, code quality reviews, weekly check-ins, and professional development throughout the 60-day engagement.

INDUSTRY GUIDE:

Aryan Sharma

REPORTING MANAGER:

Sonia Sharma

REVIEW CADENCE:

Bi-Weekly Performance Review (Every 14 Days)

COMMUNICATION CHANNEL:

Email / WhatsApp Business (official channel)

QUERY RESPONSE SLA:

24 Hours (working days); immediate for critical blockers

WEEKLY REPORT DUE:

Every Friday, 6:00 PM IST — hr@nextgenforgetechnologies.com

Review	Week	Agenda	Output
Week 2 Check-in	2	Project setup, all 12+ schemas, auth APIs, RBAC middleware progress review	Schema sign-off, early feedback
Month 1 Mid-Review	4	Warehouse, Product, Inventory, Customer, Order, Shipment APIs — live Postman demo	M1 mid-point evaluation, adjustments
Month 1 Final Review	4 End	Vehicle, Driver, Dispatch, Analytics, Notifications, Ticket APIs — full backend demo	Backend sign-off, Month 2 plan
Month 2 Mid-Review	6	Auth UI, Dashboard, Warehouse/Product/Inventory/Customer/Order React screens	UI quality feedback, UX adjustments
Final Evaluation	8	Complete platform demo: all 5 roles, Dispatch UI, Reports, live deployment verification	Final scorecard, certificate eligibility

10

EVALUATION CRITERIA

Performance will be evaluated at the conclusion of the internship by the Industry Guide and reviewed by the Reporting Manager. Bi-weekly review scores contribute cumulatively to the final evaluation scorecard.

Parameter	Weight	Assessment Criteria
Technical Implementation	22%	Correctness, completeness, and functionality of all 15 modules across backend and frontend.
API & Database Design	18%	Quality of MongoDB schemas, REST API design consistency, and aggregation pipeline accuracy.
Frontend UI & User Experience	15%	React component quality, Tailwind design, responsiveness, role-based rendering, and Chart.js integration.

Parameter	Weight	Assessment Criteria
Task Completion & Timeliness	15%	Percentage of weekly 2+2+1 task blocks completed on schedule and to the required quality standard.
Code Quality	12%	Readability, modular structure, inline documentation, and adherence to RESTful and React engineering standards.
Deployment & Architecture	10%	Successful live deployment on Vercel, Render, and Atlas with all environment variables correctly configured.
Professionalism	8%	Quality of weekly progress reports, mentor review presentations, and communication responsiveness.
TOTAL	100%	Holistic evaluation across all technical, architectural, and professional dimensions.

11

EXPECTED LEARNING OUTCOMES

Upon successful completion of this internship, the candidate will have developed the following practical competencies applicable to professional Full Stack enterprise engineering roles:

01	Full-Stack Enterprise Architecture	End-to-end proficiency in building production-ready modular MERN applications with role-based access control and 15+ integrated operational modules.
02	Advanced MongoDB Engineering	Practical expertise in designing complex document schemas with inter-document references, writing MongoDB aggregation pipelines, and optimising indexed queries.
03	Secure REST API Development	Ability to design secure, scalable, role-protected Express.js APIs with JWT authentication, Express Validator input validation, and comprehensive audit logging.
04	Professional React Development	Proficiency in building complex, role-aware React SPAs with React Context, React Router v6 protected routes, Axios interceptors, and Recharts data visualizations.
05	Cloud Deployment Operations	Hands-on experience deploying decoupled MERN architectures to free-tier production platforms: MongoDB Atlas, Render (backend), and Vercel (frontend) with environment-variable management.
06	Professional Engineering Practices	Demonstrated competency in weekly deliverables, structured API documentation, Postman test collection management, GitHub version control with branching, and formal mentor progress reporting.

ACADEMIC INTEGRITY & PROFESSIONAL ETHICS



Candidates must maintain high standards of academic honesty throughout this engagement. Submitting AI-generated or externally copied code without understanding its functionality is strictly prohibited. The candidate must be able to independently explain all architectural decisions, MongoDB schema relationships, API design choices, RBAC middleware logic, and deployment configuration during the final evaluation session. Violation of this policy will result in immediate disqualification from certification.

Dear Aryan,

Welcome to the NextGen Forge Technologies Software Development Internship Program. This assignment has been thoughtfully designed to be achievable at **3–5 hours per day, Monday to Friday** — delivering genuine, portfolio-worthy full-stack industry experience within a focused, structured 60-day (8-week) engagement.

Over the next 8 weeks, you will design, build, and deploy the **LogiFlow Enterprise Logistics Operations & Warehouse Management Platform** — a complete, production-grade enterprise application supporting five distinct user roles across a unified platform. This is exactly the type of real-world full-stack project that demonstrates job-ready engineering capability to prospective employers in enterprise logistics, supply chain, and cloud software development.

Approach each sprint with **curiosity** — explore how real-world logistics and warehouse platforms solve the same challenges at scale; **discipline** — commit your daily hours and honour weekly sprint commitments; and **ownership** — treat every REST API endpoint, every React component, and every MongoDB document schema as a reflection of your professional engineering identity.

Your Industry Guide **Aryan Sharma** and Reporting Manager **Sonia Sharma** are here to support your technical and professional growth throughout this journey. Raise blockers early, communicate proactively, and never allow an unresolved issue to stall your progress beyond 24 hours.

By completing this internship, you will carry with you a production-deployed full-stack portfolio project, verifiable enterprise software engineering experience, and the technical confidence to pursue advanced full-stack developer roles with credibility. The work you deliver here is real — treat it accordingly.

We look forward to seeing your platform live. Welcome to the NextGen Forge family.

— The People & Engineering Team
NextGen Forge Technologies, Pune, Maharashtra

Issued By:

NextGen Forge Technologies

HR & People Operations

hr@nextgenforgetechnologies.com

Industry Guide:

Aryan Sharma

Full Stack Development Lead

Document Reference:

NFGT/HR/INT/2026/160

Date Issued: 22 July 2026